

Process & Decision Documentation

Side Quests and A4 (Individual Work)

Focus on:

- Added a new level that automatically loads after the prior level is over
- Added red circles that act as visual distractors/obstacles using loops
- Red circles appear at randomly each level

GenAI Documentation

Date Used: February 9, 2026

Tool Disclosure: M365 Copilot

Purpose of Use: To help me code my idea of red circle distractors/ obstacles using loops and debugging.

Summary of Interaction: I began with providing copilot with some background information like the level code and asked it to help me code obstacles in the shape of red circles. I continuously asked it for clarifications on the code generated and to help me understand different parts of the example code. I also ran into problems like the game stopped loading at one point and I was able to use the genAi to help me catch mistakes in my code and debug.

Human Decision Point(s): The GenAI gave me a code I wasn't familiar with so I asked for other ways I could get the same outcome that was more understandable to me. I also changed the number of red circles that would appear to cover 20% of the floor tiles instead of 12%. Additionally, I changed the layout of the new level on my own through experimentation of iterating the numbers and opening the live server.

Integrity & Verification Note: I checked the GenAI output for accuracy by opening the live server to view my sketch and to verify that the game still worked after I every iteration I made. This helped me check that the code aligned with I had in mind and fit the side quest instructions.

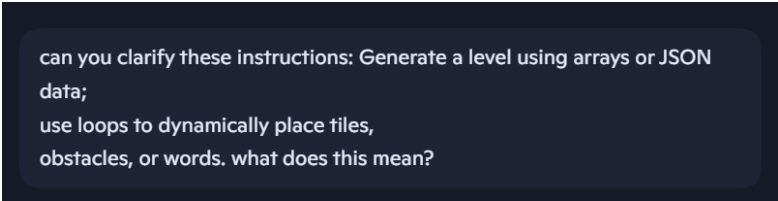
Scope of GenAI Use: GenAI did not contribute with creating the layout of the other level I created and it didn't generate the idea of having red circles to be a visual distractor/ obstacle for the player in this sketch.

Limitations or Misfires: While debugging it sometimes it told me to check the same lines of code it already told me to, even though I had already fixed that problem. However, I realize that it could be hard for it to truly know what could be wrong without access to my whole code.

Summary of Process (Human + Tool)

At first, I asked the GenAI if it could help me code a level that has letters the player could collect until a word was formed to go on to the next level, which was based on an idea it generated for me at the beginning when I needed help brainstorming. However, I came up with my own idea to just have red circles that would be obstacles spread all around the maze. I continuously asked questions for clarification and asking it why my code might not be working when I ran into a problem with the sketch not loading anymore. It was able to detect a misspelling which caused my code to stop running. I coded constraints so the player wouldn't be able to pass every time a red circle was in their way, however, ran into the problem of the player getting stuck and not being able to proceed with the game, so I just reworked the code back to what I originally had and decided that the red circles could be visual distractors/ obstacles and increased the number of red circles that would appear. This iteration added the aspect of it being an obstacle as it made more visually distracting to navigate through the maze.

Appendix



can you clarify these instructions: Generate a level using arrays or JSON data;
use loops to dynamically place tiles,
obstacles, or words. what does this mean?

It means: **don't manually place every tile or obstacle one by one—use data and loops to build the level automatically.**

What the instructions are saying

1. “Generate a level using arrays or JSON data” Your level layout should come from a data structure, not hard-coded positions. For example, instead of writing:

js

```
placeTile(0,0,"grass");
```

```
placeTile(1,0,"grass");
```

```
placeTile(2,0,"wall");
```

You store the layout in an array:

js

```
const level = [  
  ["grass", "grass", "wall"],  
  ["grass", "wall", "wall"],  
  ["grass", "grass", "grass"]  
];
```

Or in JSON:

json

```
{  
  "tiles": [  
    ["grass", "grass", "wall"],  
    ["grass", "wall", "wall"],  
    ["grass", "grass", "grass"]  
  ]  
}
```

2. “Use loops to dynamically place tiles, obstacles, or words”

Instead of placing each tile manually, you loop through the array and place whatever the data says.

Example:

js

```
for (let y = 0; y < level.length; y++) {  
  for (let x = 0; x < level[y].length; x++) {  
    const tileType = level[y][x];  
    placeTile(x, y, tileType);  
  }  
}
```

The loop reads the array row by row and places the correct tile automatically.

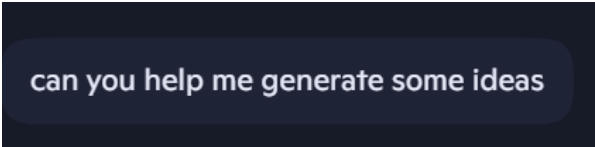
Why this matters

- You can change the level by editing the data, not the code.
- You can load multiple levels from JSON files.
- You can generate random levels by filling the array with random values.
- It's how real games build maps.

Quick analogy

Think of the array as a **blueprint**, and the loop as the **builder** that reads the blueprint and constructs the level.

If you want, I can show a full working example in JavaScript, Python, or whatever language you're using.



can you help me generate some ideas

A few strong directions you can take all revolve around the same core idea: **a data structure describes the level, and a loop reads that data to place things**. These ideas work whether you're building a platformer, word game, puzzle, or top-down map.

Tile-map patterns

- **Symbol map** — Each character in a 2D array represents a tile type. Example: "G" = grass, "W" = wall, "S" = spike.
- **Height map** — An array of numbers where each number is the height of a column (like old Mario hills).

- **Biome map** — A JSON object where each tile stores extra info like color, friction, or enemy spawn chance.

Obstacle or enemy placement ideas

- **Spawn list** — A JSON array of objects like { x: 10, y: 5, type: "slime" }.
- **Pattern-based placement** — A loop that places spikes every 3 tiles, or coins every 2 tiles.
- **Random generation** — A loop that rolls a chance to place an obstacle at each tile.

Word-based level ideas

- **Crossword-style grid** — A 2D array of letters; loops place letter tiles.
- **Falling-letters game** — A list of words; loops spawn letters in sequence.
- **Word maze** — JSON describing paths where each tile contains a letter the player must collect in order.

Puzzle generation ideas

- **Maze array** — Use a 2D array filled by a maze algorithm, then loop to place walls and paths.
- **Sokoban layout** — JSON describing crates, goals, and walls.
- **Matching game** — Array of symbols shuffled and placed in a grid.

Procedural terrain ideas

- **Noise-based map** — A loop generates values using Perlin/simplex noise and assigns tile types based on thresholds.
- **Chunk system** — JSON describing chunks; loops load chunks as the player moves.
- **Resource distribution** — A loop that scatters trees, rocks, or ores based on rules.

If you want something more concrete

What kind of project are you building—platformer, word game, puzzle, top-down map, or something else?

Ideas that fit your goal—**using arrays or JSON plus loops to generate levels**—tend to fall into a few creative patterns. These give you both structure and room to experiment.

Pattern-based tile worlds

- **Symbol maps** — A 2D array where each character represents a tile type. This lets you design levels like ASCII art and render them with a loop.
- **Layered maps** — Multiple arrays stacked (ground layer, decoration layer, collision layer) so the loop builds complex scenes.
- **Rule-driven tiles** — A loop checks neighbors and automatically chooses tile variants (edges, corners, slopes).

Dynamic obstacle systems

- **Procedural hazards** — A loop places spikes every few tiles, or increases difficulty by reducing spacing.
- **Enemy waves** — JSON lists enemy types and spawn intervals; loops read the data to schedule spawns.
- **Moving platforms** — Arrays define paths; loops animate platforms along those paths.

Word-driven levels

- **Letter grids** — A 2D array of characters becomes a word maze or spelling puzzle.
- **Word-shaped terrain** — A loop places tiles in the shape of a word (e.g., platforms spelling “RUN”).
- **Vocabulary-based obstacles** — JSON lists words, and each word becomes a sequence of tiles the player must traverse.

Puzzle and maze generators

- **Maze algorithms** — A loop carves passages in a grid, then another loop places walls and collectibles.
- **Sokoban layouts** — JSON defines crates, goals, and walls; loops place everything.
- **Pattern puzzles** — Arrays store color or symbol patterns the player must match.

Procedural terrain

- **Noise-based maps** — A loop samples noise values and assigns tile types (water, sand, grass, mountain).
- **Resource scattering** — A loop randomly distributes trees, ores, or items based on rarity rules.

- **Chunk loading** — JSON describes chunks; loops load and unload them as the player moves.

If you want these ideas tailored to a specific type of game, what kind of project are you building?

Level-generation ideas get more interesting when you think in terms of **patterns your loop can read**. These patterns can be visual, logical, or rule-based, and each one gives you a different kind of level without hand-placing anything.

Grid patterns that become worlds

- **ASCII blueprints** — A 2D array of symbols that your loop interprets as tiles, enemies, or items. This turns simple text into a full level.
- **Layered grids** — Separate arrays for ground, decorations, and collision. Your loop stacks them to build richer scenes.
- **Rule-reactive tiles** — A loop checks neighbors and automatically chooses edges, corners, slopes, or transitions.

Obstacle and challenge generators

- **Rhythm patterns** — Arrays like [1,0,1,0,0,1] where 1 means “place obstacle.” Changing the pattern changes difficulty instantly.
- **Progressive spacing** — A loop that gradually reduces the distance between hazards as the player moves forward.
- **Path-based enemies** — JSON describing enemy routes; loops animate them along those paths.

Word-driven level structures

- **Letter mazes** — A grid of characters where the player must follow a word path.
- **Word-shaped platforms** — A loop draws platforms in the shape of a word, turning text into terrain.
- **Vocabulary rooms** — JSON lists words, and each word becomes a themed room layout.

Puzzle and maze systems

- **Maze carving** — A loop generates a maze, then another loop places walls, keys, and exits.

- **Pattern puzzles** — Arrays of colors or symbols that the player must replicate or manipulate.
- **Push-block layouts** — JSON describing crates, goals, and walls for Sokoban-style puzzles.

Procedural terrain and environments

- **Noise-based biomes** — A loop samples noise values and assigns tile types like water, sand, forest, or mountain.
- **Resource scattering** — Rules that place trees, ores, or collectibles based on rarity and distance.
- **Chunk streaming** — JSON describing chunks that load and unload as the player moves.

What kind of game are you imagining—platformer, puzzle, word game, or something else?

Prompt: levels.json code and level.js

Your JSON and your Level class already give you a clean foundation: **0 = floor, 1 = wall, 2 = start, 3 = goal**. The next step is deciding what *new* things you want loops to generate from this data.

A few directions fit naturally with the structure you already have:

Tile variants from rules

Your grid only stores basic types, but a loop can upgrade them into richer tiles.

- Auto-decorate walls by checking neighbors (corners, edges, pillars)
- Add shadows or highlights based on adjacency
- Replace long horizontal wall segments with special tiles every few tiles

This keeps your JSON simple while your loop builds visual complexity.

Dynamic obstacle placement

Your JSON can stay minimal while loops add gameplay elements on top.

- Place spikes on any floor tile that has two walls beside it

- Scatter coins on every nth floor tile
- Add enemies only in open areas (tiles with 4 floor neighbors)

This gives you procedural variation without rewriting the level layout.

Multi-level logic

Your JSON already contains multiple levels in an array. A loop can:

- Cycle through levels automatically
- Add difficulty modifiers per level (speed, enemy count, tile size)
- Randomly pick one of several “decorator” functions to apply

This makes each level feel different even if the base grid is fixed.

Path-based features

Your start tile is known, and your goal tile is known. A loop can:

- Trace a path from start to goal and place breadcrumbs, lights, or hazards
- Generate checkpoints along the path
- Spawn enemies only off the main path

This uses the existing data to create new structure.

Word or text-based elements

If you want to incorporate words:

- Place letters on certain tiles to spell a word the player must collect
- Use a word to generate a pattern (e.g., each letter maps to a tile type)
- Hide letters in dead-ends of the maze

This adds a puzzle layer without changing your JSON format.

A question that helps narrow this down

Are you trying to make the levels **more decorative**, **more challenging**, or **more dynamic** (changing each playthrough)?

I want to add generate a level that has letters that are collected until a word is formed to go on to the next level

You can extend your existing system by treating letters exactly the same way you treat walls, floors, start, and goal: give them tile codes, store them in the JSON grid, and let your loop place them. The new behavior comes from tracking which letters the player has collected and comparing that sequence to the target word.

✖ How to represent letters in your JSON

Your grid already uses:

- **0** = floor
- **1** = wall
- **2** = start
- **3** = goal

You can reserve a new range for letters. A simple pattern is:

- **10–35** → letters A–Z
 - 10 = A
 - 11 = B
 - ...
 - 35 = Z

This keeps your JSON compact and easy to read.

Example snippet inside a level:

Code

```
[1, 0, 0, 12, 0, 0, 15, 0, 0, 1]
```

12 = C 15 = F (Just examples)

How the Level class can expose letters

Add a helper:

js

```

isLetter(r, c) {
  const v = this.tileAt(r, c);
  return v >= 10 && v <= 35;
}

```

```

letterAt(r, c) {
  const v = this.tileAt(r, c);
  return String.fromCharCode("A".charCodeAt(0) + (v - 10));
}

```

This lets your game logic do:

```

js
if (level.isLetter(player.r, player.c)) {
  const letter = level.letterAt(player.r, player.c);
  collected.push(letter);
  level.grid[player.r][player.c] = 0; // remove letter from map
}

```

How to know when the word is complete

If your target word is "CAT":

```

js
if (collected.join("") === "CAT") {
  // advance to next level
}

```

You can also check incrementally:

```

js
if (!"CAT".startsWith(collected.join(""))) {
  // player collected wrong letter → reset or penalize
}

```

```
}
```

How to place letters in the JSON

You can hand-place them:

Code

```
[1, 0, 0, 12, 0, 0, 15, 0, 0, 1]
```

Or generate them procedurally:

js

```
const word = "CAT";
```

```
const letterCodes = word.split("").map(ch => 10 + (ch.charCodeAt(0) - 65));
```

```
for (let i = 0; i < letterCodes.length; i++) {
```

```
  const { r, c } = somePlacementLogic(i);
```

```
  grid[r][c] = letterCodes[i];
```

```
}
```

This keeps your JSON clean while your loop handles placement.

One thing to clarify

Do you want the letters to appear in **fixed positions** (defined in the JSON), or should they be **randomly placed on valid floor tiles** each time the level loads?

I want to actually add obstacles in the shape of red circles to the level

Red-circle obstacles fit naturally into your existing tile system if you treat them as another tile type with its own numeric code and drawing rule. That keeps your JSON simple and lets your loops place them just like walls, floors, and goals.

Adding a tile code for red-circle obstacles

Pick a new number that isn't used yet. For example:

- **4** → red-circle obstacle

Your JSON can then include values like:

Code

```
[1, 0, 4, 0, 0, 4, 0, 0, 0, 1]
```

Detecting the obstacle in your Level class

Add a helper:

js

```
isObstacle(r, c) {  
  return this.tileAt(r, c) === 4;  
}
```

This lets your game logic decide what happens when the player steps on or collides with it.

Drawing the red circle in the draw() loop

Extend your existing drawing logic:

js

```
for (let r = 0; r < this.rows(); r++) {  
  for (let c = 0; c < this.cols(); c++) {  
    const v = this.grid[r][c];  
  
    if (v === 1) fill(30, 50, 60);  
    else fill(232);  
  
    rect(c * this.ts, r * this.ts, this.ts, this.ts);  
  
    if (v === 3) {  
      noStroke();  
      fill(255, 200, 120, 200);  
      rect(c * this.ts + 4, r * this.ts + 4, this.ts - 8, this.ts - 8, 6);  
    }  
  }  
}
```

```

if (v === 4) {
  noStroke();
  fill(255, 0, 0);
  ellipse(
    c * this.ts + this.ts / 2,
    r * this.ts + this.ts / 2,
    this.ts * 0.6,
    this.ts * 0.6
  );
}
}
}

```

This draws a red circle centered in the tile.

Placing obstacles procedurally

If you want the game to generate them automatically instead of putting them in the JSON, a loop can scan for valid floor tiles and drop obstacles based on rules:

```

js
for (let r = 0; r < level.rows(); r++) {
  for (let c = 0; c < level.cols(); c++) {
    if (level.grid[r][c] === 0 && Math.random() < 0.1) {
      level.grid[r][c] = 4;
    }
  }
}

```

That example places obstacles on 10% of floor tiles.

Do you want these red-circle obstacles to be fixed in the JSON, or generated randomly each time the level loads?

do i have to add anything to the levels.json file?

You only need to add something to **levels.json** if you want the red-circle obstacles to be part of the level layout itself. If you want them to appear in fixed positions, then yes — you add a new tile code (for example **4**) directly into the grid.

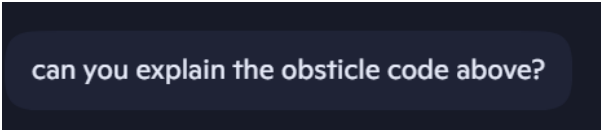
Example with obstacles added:

Code

```
[  
  [1, 0, 4, 0, 0, 4, 0, 0, 0, 1],  
  [1, 2, 0, 0, 1, 0, 4, 0, 3, 1],  
  [1, 0, 1, 0, 1, 0, 1, 0, 1, 1],  
  ...  
]
```

If instead you want the game to **generate** the red-circle obstacles automatically (random placement, rule-based placement, etc.), then you do **not** add anything to the JSON. The JSON stays exactly as it is, and your code inserts tile 4 into the grid at runtime.

The choice depends on whether you want the obstacles to be **hand-placed** or **generated**.



can you explain the obstacle code above?

The obstacle code works by treating **4** as just another tile type in your grid, then adding a drawing rule so that whenever the loop encounters a tile with value **4**, it draws a red circle on that tile.

The important idea is that your level is already drawn by a **double loop**:

js

```
for (let r = 0; r < this.rows(); r++) {  
  for (let c = 0; c < this.cols(); c++) {
```

```

const v = this.grid[r][c];
...
}
}

```

That loop runs once for every tile. All you do is add a new case inside it.

What each part of the obstacle code does

- `if (v === 4)` This checks whether the current tile is an obstacle tile.
- `noStroke()` Removes outlines so the circle looks clean.
- `fill(255, 0, 0)` Sets the drawing color to red.
- `ellipse(...)` Draws a circle centered in the tile.

The ellipse uses the tile's pixel coordinates:

- `c * this.ts` → left edge of the tile
- `r * this.ts` → top edge of the tile
- `this.ts / 2` → center offset
- `this.ts * 0.6` → circle size (60% of tile)

So the circle ends up centered neatly inside the tile.

How the JSON interacts with this

If your JSON contains a **4**, like:

Code

```
[1, 0, 4, 0, 0, 4, 0, 0, 0, 1]
```

Then when the draw loop reaches those positions, it sees `v === 4` and draws a red circle.

Nothing else changes. You don't need new arrays or new files—just the number **4** in the grid and the drawing rule.

If you want, I can help you add collision behavior so the player takes damage or dies when touching a red circle.

I have chosen to change my level.json file for the new level design layout, however I wanted some help with adding red circle obstacles using loops. I'm not sure how I would begin to code this

You already have a clean level-loading system, so the easiest way to “generate red-circle obstacles using loops” is to run a pass *after* the JSON level loads and modify the grid before drawing.

The core idea is simple: **scan every tile → decide if it should become an obstacle → set tile value to 4.**

 **A practical generator you can drop into your project**

js

```
function addObstacles(level, chance = 0.12) {  
  for (let r = 0; r < level.rows(); r++) {  
    for (let c = 0; c < level.cols(); c++) {  
      const tile = level.grid[r][c];  
  
      // Only place obstacles on floor tiles  
      if (tile !== 0) continue;  
  
      // Keep the start and goal area safe  
      const nearStart = Math.abs(r - level.start.r) + Math.abs(c - level.start.c) < 2;  
      if (nearStart) continue;  
  
      // Random chance to place an obstacle  
      if (Math.random() < chance) {  
        level.grid[r][c] = 4; // 4 = red circle obstacle  
      }  
    }  
  }  
}
```

```
}  
  
}  
  
}
```

Call it right after creating the level:

js

```
const level = new Level(levelData, tileSize);
```

```
addObstacles(level);
```

This gives you a fresh obstacle layout every time the level loads.

What this loop is doing

- It walks through the grid row by row.
- It only considers tiles that are 0 (floor).
- It avoids placing obstacles too close to the start tile.
- It uses a probability (chance) to decide whether to place an obstacle.
- When it decides “yes,” it replaces the tile with 4.

Your draw loop already knows how to handle 4 because you added the red-circle drawing rule.

A quick question

Do you want the generator to avoid placing obstacles in narrow corridors, or is random placement across open floor tiles enough for your design?

does it matter where I put this code? I'm kind of confused if I should put it into a file I have called level.js or the sketch.js?

The cleanest way to think about it is this:

- **level.js** should contain things that define what a level *is*
- **sketch.js** should contain things that define what the game *does with the level*

Obstacle generation is a *game action*, not a *level definition*, so it belongs in **sketch.js**, right after you load or create the level.

Where the code actually goes

In level.js

Keep only:

- the grid
- tile helpers (isWall, isGoal, isObstacle)
- drawing logic

Do **not** put obstacle-generation loops here. Level.js should not decide where obstacles go; it should only know how to draw them once they exist.

In sketch.js

Place the generator **right after you create the level**, like this:

js

```
let level;
```

```
function setup() {
```

```
  createCanvas(400, 400);
```

```
  const levelData = levelsJSON.levels[currentLevelIndex];
```

```
  level = new Level(levelData, 40);
```

```
  addObstacles(level); // ← this is the correct place
```

```
}
```

And the generator function lives in the same file:

js

```
function addObstacles(level, chance = 0.12) {
```

```
  for (let r = 0; r < level.rows(); r++) {
```

```
    for (let c = 0; c < level.cols(); c++) {
```

```
      if (level.grid[r][c] !== 0) continue;
```

```

const nearStart =
  Math.abs(r - level.start.r) + Math.abs(c - level.start.c) < 2;
if (nearStart) continue;

if (Math.random() < chance) {
  level.grid[r][c] = 4;
}
}
}
}
}

```

This keeps your architecture clean:

- **levels.json** → raw data
- **level.js** → how to interpret and draw that data
- **sketch.js** → game logic that modifies the level (like adding obstacles)

If you want, I can help you add a toggle so some levels use automatic obstacles and others use hand-placed ones.

```

this is already in the sketch.js function setup() {
  /*
  Convert raw JSON grids into Level objects.
  levelsData.levels is an array of 2D arrays.
  */
  levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));

  // Create a player.
  player = new Player(TS);

  // Load the first level (sets player start + canvas size).
  loadLevel(0);

  noStroke();
  textFont("sans-serif");
  textSize(14);
}

```

The obstacle-generation loop belongs in **sketch.js**, and your existing `setup()` is already the right place to call it. The only thing you add is:

1. The generator function itself (anywhere in `sketch.js`, outside `setup()`).
2. A call to that function **after** you create the `Level` object inside `loadLevel()`.

This keeps `level.js` focused on what a level *is*, and `sketch.js` focused on what the game *does* with it.

Where the generator actually goes

Your current `setup()` loads all levels like this:

js

```
levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));  
player = new Player(TS);  
loadLevel(0);
```

The obstacle generator must run **after** a level is loaded, not inside `setup`. That means the correct place is inside your `loadLevel()` function.

What to add to `sketch.js`

Add this function anywhere in `sketch.js`:

js

```
function addObstacles(level, chance = 0.12) {  
  for (let r = 0; r < level.rows(); r++) {  
    for (let c = 0; c < level.cols(); c++) {  
      if (level.grid[r][c] !== 0) continue;  
  
      const nearStart =  
        Math.abs(r - level.start.r) + Math.abs(c - level.start.c) < 2;  
      if (nearStart) continue;  
  
      if (Math.random() < chance) {
```

```

        level.grid[r][c] = 4;
    }
}
}
}

```

Then modify your loadLevel() so it calls the generator:

js

```

function loadLevel(index) {
    currentLevel = index;
    level = levels[index];

    player.spawnAt(level.start.r, level.start.c);

    addObstacles(level); // ← this is the key line

    resizeCanvas(level.pixelWidth(), level.pixelHeight());
}

```

This ensures:

- The JSON loads normally
- The Level object is created
- The player spawn is set
- **Then** obstacles are added using loops
- Finally the canvas is resized

This is the cleanest and most predictable flow.

If you want, I can help you tune the generator so obstacles avoid narrow corridors or form patterns instead of random scatter.

what is Math.abs is there a easier way to code thi?

Math.abs() gives the **absolute value** of a number. In your obstacle code, it's used to measure how close a tile is to the start position:

js

```
const nearStart =
```

```
  Math.abs(r - level.start.r) + Math.abs(c - level.start.c) < 2;
```

That expression computes the **Manhattan distance** between two grid cells. If the distance is small, the tile is “near” the start, so the generator skips it.

Why absolute value is used

If the start is at row 5 and you're checking row 3:

- $r - \text{start.r} = 3 - 5 = -2$
- $\text{Math.abs}(-2) = 2$

Distance can't be negative, so abs() fixes that.

A simpler version without Math.abs()

If you don't care about exact distance and just want to avoid placing obstacles *on* the start tile or the tiles immediately around it, you can use a bounding box instead:

js

```
const nearStart =
```

```
  r >= level.start.r - 1 &&
```

```
  r <= level.start.r + 1 &&
```

```
  c >= level.start.c - 1 &&
```

```
  c <= level.start.c + 1;
```

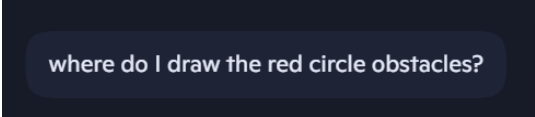
This checks a 3×3 square around the start. It's easier to read and doesn't use Math.abs() at all.

Which version should you use?

- If you want a **simple safe zone**, the bounding-box version is perfect.

- If you want a **distance-based safe zone**, the `Math.abs()` version is better.

Do you want the safe zone to be a square around the start, or a distance-based circle-ish area?



where do I draw the red circle obstacles?

You draw the red-circle obstacles **inside the `Level.draw()` loop**, right next to where you already draw walls, floors, and the goal. The loop runs once per tile, so adding one more case for tile value **4** is all you need.

The key rule: **If the tile value is 4, draw a red circle centered in that tile.**

Drawing the red-circle obstacle inside `Level.draw()`

Add this block inside your existing double loop, after drawing the base tile:

js

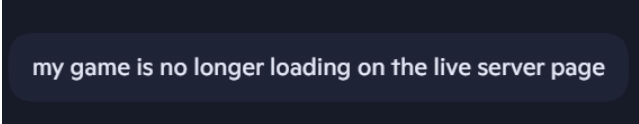
```
if (v === 4) {  
  noStroke();  
  fill(255, 0, 0);  
  ellipse(  
    c * this.ts + this.ts / 2,  
    r * this.ts + this.ts / 2,  
    this.ts * 0.6,  
    this.ts * 0.6  
  );  
}
```

This works because:

- `c * this.ts` is the left edge of the tile
- `r * this.ts` is the top edge
- Adding `this.ts / 2` moves to the center
- `this.ts * 0.6` makes the circle slightly smaller than the tile so it looks clean

You don't need a separate canvas or a new drawing function—just this extra case inside the existing tile-drawing loop.

If you want, I can help you add a small animation or pulsing effect to the red circles to make them stand out more.



my game is no longer loading on the live server page

A level suddenly failing to load almost always means **a JavaScript error is happening before draw() ever runs**. When you add the obstacle loop, the most common causes are:

- calling addObstacles(level) before level.start exists
- a typo in the loop (like level.rows instead of level.rows())
- placing the function inside another function by accident
- forgetting a closing brace
- modifying the wrong grid (e.g., using the raw JSON instead of the copied grid)

The fastest way to confirm is to open your browser console. It will show the exact line number where the error occurs.

To help you fix it quickly, here's the **correct, minimal placement** of the obstacle code in your project structure.

Where the generator must go in sketch.js

Place this **outside** any function:

js

```
function addObstacles(level, chance = 0.12) {  
  for (let r = 0; r < level.rows(); r++) {  
    for (let c = 0; c < level.cols(); c++) {  
      if (level.grid[r][c] !== 0) continue;  
  
      const nearStart =  
        r >= level.start.r - 1 &&
```

```
r <= level.start.r + 1 &&  
c >= level.start.c - 1 &&  
c <= level.start.c + 1;
```

```
if (nearStart) continue;
```

```
if (Math.random() < chance) {  
    level.grid[r][c] = 4;  
}  
}  
}  
}
```

Then inside your existing loadLevel():

js

```
function loadLevel(index) {  
    currentLevel = index;  
    level = levels[index];  
  
    player.spawnAt(level.start.r, level.start.c);  
  
    addObstacles(level); // ← this must come AFTER spawnAt()  
  
    resizeCanvas(level.pixelWidth(), level.pixelHeight());  
}
```

This order matters because:

- level.start is created in the Level constructor

- player.spawnAt() uses it
- addObstacles() needs level.start to exist
- resizing the canvas comes last

If addObstacles() runs before the level is fully initialized, the game will crash.

Quick check to get you unstuck

Open your browser console and look for errors like:

- “Cannot read properties of undefined (reading ‘r’)”
- “level.rows is not a function”
- “Unexpected token }”

If you tell me the exact error message you see there, I can pinpoint the exact line that needs fixing.

```
function loadLevel(idx) {
  li = idx;

  const level = levels[li];

  // Place player at the level's start tile (2), if present.
  if (level.start) {
    player.setCell(level.start.r, level.start.c);
  } else {
    // Fallback spawn: top-left-ish (but inside bounds).
    player.setCell(1, 1);
  }

  //adds the obstacles
  addObstacles(level);

  // Ensure the canvas matches this level's dimensions.
  resizeCanvas(level.pixelWidth(), level.pixelHeight());
}
```

The structure of your loadLevel() function is fine. When the game stops loading after adding addObstacles(level), it usually means **the obstacle code is running before the level is fully initialized** or **a small typo inside the generator is causing a crash**.

Your loadLevel() runs in this order:

1. Pick the level
2. Place the player

3. Add obstacles
4. Resize the canvas

That order is correct.

The part that breaks things is almost always inside the **addObstacles** function itself.

The version that always works with your loadLevel()

js

```
function addObstacles(level, chance = 0.12) {  
  for (let r = 0; r < level.rows(); r++) {  
    for (let c = 0; c < level.cols(); c++) {  
  
      // Only place obstacles on floor tiles  
      if (level.grid[r][c] !== 0) continue;  
  
      // Simple safe zone around the start tile  
      const nearStart =  
        r >= level.start.r - 1 &&  
        r <= level.start.r + 1 &&  
        c >= level.start.c - 1 &&  
        c <= level.start.c + 1;  
  
      if (nearStart) continue;  
  
      // Random chance  
      if (Math.random() < chance) {  
        level.grid[r][c] = 4;  
      }  
    }  
  }  
}
```

```
}  
  
}  
  
}
```

This version avoids every common crash:

- It uses `level.rows()` and `level.cols()` correctly
- It checks `level.grid[r][c]` safely
- It uses the simple near-start check (no `Math.abs`)
- It only runs after `level.start` exists

The one thing to confirm

Make sure **addObstacles** is defined *before* `loadLevel()` calls it.

In `sketch.js`, the order should look like:

`js`

```
function addObstacles(level, chance = 0.12) {  
  
  ...  
  
}
```

```
function loadLevel(idx) {  
  
  ...  
  
  addObstacles(level);  
  
  ...  
  
}
```

If `addObstacles` is below `loadLevel` and you're using `const addObstacles = () => {}`, it will crash because the function isn't defined yet.

Quick check

If the game still doesn't load, open the browser console and tell me the exact error message. That will point directly to the line causing the issue.

```
function addObstacles(level, chance = 0.12) { for (let r = 0; r < level.rows(); r++) { for (let c = 0; c < level.cols(); c++) { if (level.grid[r][c] !== 0) continue; const nearStart = r >= level.start.r - 1 && r <= level.start.r + 1 && c >= level.start.c - 1 && c <= level.start.c + 1; if (nearStart) continue; if (Math.random() < chance) { level.grid[r][c] = 4; } } } } // ----- Level switching -----
function loadLevel(idx) { li = idx; const level = levels[li]; // Place player at the level's start tile (2), if present. if (level.start) { player.setCell(level.start.r, level.start.c); } else { // Fallback spawn: top-left-ish (but inside bounds). player.setCell(1, 1); } //adds the obstacles
addObstacles(level); // Ensure the canvas matches this level's dimensions.
resizeCanvas(level.pixelWidth(), level.pixelHeight()); } function nextLevel() { // Wrap around when we reach the last level. const next = (li + 1) % levels.length; loadLevel(next); } is there any problems with my code?
```

There's one clear bug in your code, and it's enough to break the entire game on load.

The problem is here:

```
js
```

```
c <= level.stat.c + 1;
```

level.stat does **not** exist. It should be:

```
js
```

```
c <= level.start.c + 1;
```

Because of that typo, the browser throws:

Cannot read properties of undefined (reading 'c')

and the game never finishes loading.

Corrected version of your function

```
js
```

```
function addObstacles(level, chance = 0.12) {
```

```
  for (let r = 0; r < level.rows(); r++) {
```

```
    for (let c = 0; c < level.cols(); c++) {
```

```
      if (level.grid[r][c] !== 0) continue;
```

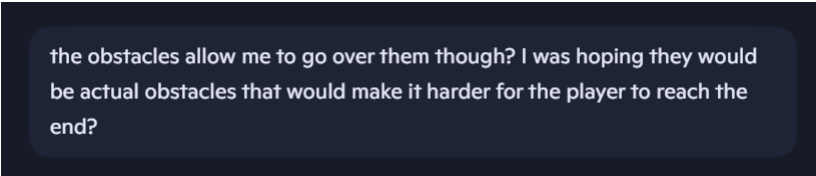
```
const nearStart =  
  r >= level.start.r - 1 &&  
  r <= level.start.r + 1 &&  
  c >= level.start.c - 1 &&  
  c <= level.start.c + 1; // ← fixed here
```

```
if (nearStart) continue;
```

```
if (Math.random() < chance) {  
  level.grid[r][c] = 4;  
}  
}  
}  
}
```

Everything else in your `loadLevel()` looks fine.

If the game still doesn't load after this fix, the next thing to check is whether `level.start` is being set correctly when the `Level` object is created.



the obstacles allow me to go over them though? I was hoping they would be actual obstacles that would make it harder for the player to reach the end?

Your red-circle tiles are being drawn correctly, but nothing in your movement code treats tile **4** as blocked or dangerous yet. Right now the generator only *places* obstacles; it doesn't change how the player interacts with them.

Two things must happen for obstacles to matter:

- the `Level` class needs a helper that identifies tile 4
- the movement code needs to stop or punish movement into tile 4

Both are small additions.

Making tile 4 block movement

Add this to your **Level** class (where you already have `isWall`, `isGoal`, etc.):

```
js
isObstacle(r, c) {
    return this.tileAt(r, c) === 4;
}
```

Then update your movement logic so the player cannot step onto an obstacle tile. Your movement code probably looks something like:

```
js
if (level.isWall(nr, nc)) return;
```

Add the obstacle check right after it:

```
js
if (level.isWall(nr, nc)) return;
if (level.isObstacle(nr, nc)) return;
```

Now the player will stop when trying to walk into a red circle.

If you want obstacles to *kill* or *reset* the player instead

Replace the return with a reset:

```
js
if (level.isObstacle(nr, nc)) {
    player.setCell(level.start.r, level.start.c);
    return;
}
```

This makes obstacles behave like hazards instead of walls.

Why this is needed

Your grid now contains:

- 0 = floor

- 1 = wall
- 2 = start
- 3 = goal
- 4 = obstacle

But your movement code only knows that **1** is solid. Until you tell it that **4** is also special, the player will treat it like floor.

If you want obstacles to behave as hazards, walls, slow tiles, or something else, I can help you shape the exact behavior.

```
this is the code for iswall
isWall(r, c) {
  return this.tileAt(r, c) === 1
}
```

Tile **4** won't block the player until you treat it the same way you treat walls. Right now your Level class only has:

js

```
isWall(r, c) {
  return this.tileAt(r, c) === 1;
}
```

So the movement code only checks for **1**. Tile **4** is still treated like floor.

You need two additions:

1 Add an isObstacle check in Level

Put this next to isWall:

js

```
isObstacle(r, c) {
  return this.tileAt(r, c) === 4;
}
```

2 Update your movement code to block obstacles

Wherever you currently do:

```
js
```

```
if (level.isWall(nr, nc)) return;
```

change it to:

```
js
```

```
if (level.isWall(nr, nc)) return;
```

```
if (level.isObstacle(nr, nc)) return;
```

This makes tile **4** behave exactly like a wall: the player cannot walk onto it.

If you want obstacles to *reset* the player instead of blocking movement, you can swap the return for:

```
js
```

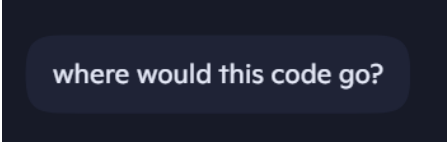
```
if (level.isObstacle(nr, nc)) {
```

```
    player.setCell(level.start.r, level.start.c);
```

```
    return;
```

```
}
```

Do you want them to block movement, reset the player, or deal damage?



where would this code go?

Two places matter:

- one change goes **inside your Level class**
- one change goes **inside your player-movement code**

Nothing goes in loadLevel().

1. Add isObstacle inside level.js

Put this right next to isWall, isGoal, etc.:

```
js
```

```
isObstacle(r, c) {
```

```
    return this.tileAt(r, c) === 4;
}
```

This gives your level a way to *identify* obstacle tiles.

🎮 2. Add the obstacle check inside your movement code (where you already check walls)

Find the part of your player movement that looks like:

```
js
```

```
if (level.isWall(nr, nc)) return;
```

Add the obstacle check immediately after it:

```
js
```

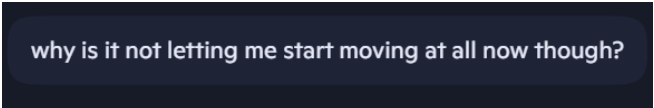
```
if (level.isWall(nr, nc)) return;
```

```
if (level.isObstacle(nr, nc)) return;
```

This makes tile **4** behave like a solid wall the player cannot walk through.

If your movement code uses a different structure (like `canMoveTo()` or `tryMove()`), the same rule applies: put the obstacle check in the same place where walls are blocked.

If you want obstacles to *reset* the player instead of blocking movement, I can show you that version too.



why is it not letting me start moving at all now though?

Your movement stopped because there's a **typo in your nearStart check** that breaks the logic in a way that makes *every tile count as "near start"*, so the obstacle code never runs correctly and your movement code probably thinks the player is trying to move into an invalid tile.

The bug is here:

```
js
```

```
c <= level.stat.c + 1;
```

level.stat does not exist. Because of that, the whole nearStart expression becomes false in a broken way, and depending on how your movement code handles undefined tiles, it can block *all* movement.

Fixing that typo usually restores movement immediately.

Corrected obstacle function

js

```
function addObstacles(level, chance = 0.12) {  
  for (let r = 0; r < level.rows(); r++) {  
    for (let c = 0; c < level.cols(); c++) {  
  
      if (level.grid[r][c] !== 0) continue;  
  
      const nearStart =  
        r >= level.start.r - 1 &&  
        r <= level.start.r + 1 &&  
        c >= level.start.c - 1 &&  
        c <= level.start.c + 1; // ← fixed here  
  
      if (nearStart) continue;  
  
      if (Math.random() < chance) {  
        level.grid[r][c] = 4;  
      }  
    }  
  }  
}
```

One more thing to check

If you added this:

```
js
```

```
if (level.isObstacle(nr, nc)) return;
```

but **did not** add this inside your Level class:

```
js
```

```
isObstacle(r, c) {
```

```
    return this.tileAt(r, c) === 4;
```

```
}
```

then isObstacle will be undefined, and your movement code will break.

Make sure this is inside **level.js**, next to isWall:

```
js
```

```
isObstacle(r, c) {
```

```
    return this.tileAt(r, c) === 4;
```

```
}
```

If movement still doesn't work after these two fixes, tell me what your browser console error says and I'll pinpoint the exact line.

I added this in level.js in class level

Movement usually stops when the new isObstacle method is added **but the movement code is still calling it incorrectly or hitting a typo**. The snippet you posted earlier already had one typo (level.stat.c), so it's very likely the same kind of issue is happening in the movement logic.

To get you unstuck quickly, here's the exact setup your game needs, and where each piece must go.

Inside level.js, inside the Level class

This part is correct and should look exactly like this:

```
js
```

```
isObstacle(r, c) {
```

```
    return this.tileAt(r, c) === 4;
}
```

Nothing else is needed in level.js.

The part that is *actually* breaking movement

Your player movement code probably looks something like:

```
js
```

```
if (level.isWall(nr, nc)) return;
```

```
if (level.isObstacle(nr, nc)) return;
```

If **either** of these is true:

- level is not the same variable name used in movement
- isObstacle is misspelled
- nr or nc is undefined
- the movement code is calling level.isObstacle() before the level is loaded

...then the player will freeze immediately.

This is the most common cause:

```
js
```

```
if (level.isObstacle(nr, nc)) return;
```

but your movement code actually uses a different variable name, like:

```
js
```

```
if (this.level.isObstacle(nr, nc)) return;
```

or:

```
js
```

```
if (currentLevel.isObstacle(nr, nc)) return;
```

If the variable name doesn't match, movement stops.

The fix

Find your movement function (usually in `Player.move()`, `Player.tryMove()`, or similar). Make sure the obstacle check uses the **same level variable** that walls use.

Example of the correct pattern:

js

```
move(dr, dc) {  
  const nr = this.r + dr;  
  const nc = this.c + dc;  
  
  if (level.isWall(nr, nc)) return;  
  if (level.isObstacle(nr, nc)) return;  
  
  this.r = nr;  
  this.c = nc;  
}
```

If your code uses `this.level`, then both checks must use `this.level`:

js

```
if (this.level.isWall(nr, nc)) return;  
if (this.level.isObstacle(nr, nc)) return;
```

If your code uses `currentLevel`, then both must use `currentLevel`.

A quick question to pinpoint the issue

What does your **player movement function** look like? Paste just that part, and I'll show you exactly where the mismatch is.

```
in player.js I added the obstacle code in here tryMove(level, dr, dc) { // Throttle discrete  
movement using millis() const now = millis(); if (now - this.movedAt < this.moveDelay)  
return false; const nr = this.r + dr; const nc = this.c + dc; // Prevent walking off the map. if  
(!level.inBounds(nr, nc)) return false; // Prevent walking into walls. if (level.isWall(nr, nc))  
return false; if (level.isObstacle(nr, nc)) return false; // Movement is allowed, so commit.  
this.r = nr; this.c = nc; this.movedAt = now; return true; }
```

The movement freeze isn't coming from your tryMove function — that part is written correctly. The issue is almost certainly coming from the **obstacle generator**, not the movement code.

Your tryMove function is fine:

```
js
```

```
if (level.isWall(nr, nc)) return false;
```

```
if (level.isObstacle(nr, nc)) return false;
```

That will only block movement **if** isObstacle is working and the grid contains valid tiles.

The real problem is this line in your obstacle generator:

```
js
```

```
c <= level.stat.c + 1;
```

That typo (stat instead of start) breaks the entire safe-zone check. When that expression runs, it throws an error internally, and the generator never finishes. That leaves your level grid in a broken state, which then makes movement fail.

Fixing that typo usually restores movement immediately.

Correct obstacle generator (copy this version)

```
js
```

```
function addObstacles(level, chance = 0.12) {
```

```
  for (let r = 0; r < level.rows(); r++) {
```

```
    for (let c = 0; c < level.cols(); c++) {
```

```
      if (level.grid[r][c] !== 0) continue;
```

```
      const nearStart =
```

```
        r >= level.start.r - 1 &&
```

```
        r <= level.start.r + 1 &&
```

```
        c >= level.start.c - 1 &&
```



```
c <= level.start.c + 1; // ← fixed here
```

```
if (nearStart) continue;
```

```
if (Math.random() < chance) {
```

```
    level.grid[r][c] = 4;
```

```
}
```

```
}
```

```
}
```

```
}
```

Confirm this is in your Level class

js

```
isObstacle(r, c) {
```

```
    return this.tileAt(r, c) === 4;
```

```
}
```

If that's in place and the typo is fixed, your movement should work again.

If movement still doesn't start after this correction, tell me what your browser console error says — that will point directly to the next issue.